Type: MCQ

Q1.

What will be the output of the following?

```
#include<stdio.h>
int main()
{
   struct     {
      int birds;
      int animals;
   }F1,*F2;
   F1.birds=1000;
   F1.animals=20;
   F2=&F1;
   printf("%d ",*F2.animals);
   return 0;  }
```

(0.5)

1. ** syntax error
2. 20
3. Garbage value
4. 1000

Q2. What does the following declaration mean?

int *ptr[10];

(0.5)

1. **ptr is array of 10 pointers to integer.
2. ptr is a pointer to an array of 10 integers.
3. ptr is an array of 10 integers.
4. ptr is an pointer to array.

Q3.What is the output of the following function considering the structure of the node and display function being called from the main function with the pointer to the beginning of the linked list?

```
struct node
{
int data;
struct node *next;
};
void display(struct node* start)
```

```
{
 if (start == NULL)
   return;
 display (start -> next) ;
 printf (" %d  ", start -> data ) ;
}
```

(0.5)

1. It will print all the nodes of linked lists
2. **It will print all the nodes of the linked list in reverse order.
3. It will print alternate nodes of the Linked List
4. It will print alternate nodes in reverse order

Q4. Consider the following code which returns a concatenated list, which contains the list pointed by ***ptr1*** followed by the list pointed by ***ptr2***. Both lists are non-empty.

```
list_pointer concatenate(list_pointer ptr1, list_pointer ptr2)
{
        list_pointer temp;
        for (temp=ptr1;temp->link; temp=temp->link)
        {
           printf("Concatenating\n");
        }
         temp->link = ptr1;
        return ptr1;
}
```

In the above code snippet, to make it work properly,

(0.5)

1. **The statement ***temp->link = ptr1;*** should be changed to ***temp->link = ptr2;***
2. The statement ***return ptr1;*** should be changed to ***return ptr2;***
3. The code is working fine.
4. The statement ***ptr2=NULL*** should be added before the return statement

Q5. A ____ is a linear collection of self-referential structures, called nodes, connected by pointer links. (0.5)

1. **Linked lists
2. Arrays
3. Stacks
4. Queues

Q6. What is the action the following function performing?

```
struct Node* function(struct Node* ptr1, struct Node* ptr2)
{
   struct Node* tmp = ptr2->next;
   ptr2->next = ptr1;
   ptr1->next = tmp;
   return ptr2;
}
```
(0.5)

1. **Swapping nodes**
2. Deleting a node
3. Inserting a node
4. Creating a newnode

Q7. What is the maximum count of parentheses that will be on the stack at any given moment while checking the balanced parenthesis in the given expression (()(())(()))? .(0.5)

1. 2
2. **3
3. 1
4. 4 OR above

Q8. Consider an infix expression: 9 + 5*(4*(7+ (1-8))). Using stack convert the expression from infix to postfix notation. What is the maximum number of symbols that will appear on the stack during the conversion of this expression? (0.5)

1. 5.
2. **8.
3. 4.
4. 7.

Q9. Consider the following code snippet of deleting an element from a linked queue. Which statement is to be added, if any, to make sure that the code works fine.

```
 int deleteq(queue_pointer *front)
{
    queue_pointer temp = *front;
    element item;
    item = temp->item;
    // Which statement to be added here ?
    free(temp);
    return item;
}
```
(0.5)

1. *rear = temp->link
2. ** *front = temp->link;
3. Nothing to be added

4. *front=NULL

Q10. Consider the singly linked list representation of polynomial and the following attach function for adding the new terms to the polynomial. Each node contains three fields coef, expon and a pointer to next node in the list.

```
void attach(int c, int e, poly_pointer *ptr)
{
   poly_pointer temp= (poly_pointer) malloc(sizeof(poly_pointer));
   temp->coef = c;
   temp->expon = e;
   (*ptr)->link = temp;
   *ptr = temp;
}
```

If ptr is pointing to the first node in the list, then the above code snippet,

(0.5)

1. ** Adds a new node to the beginning of present list
2. Adds a new node to the end of present list
3. Adds a new node at the middle of the present list
4. None of the above

Type: DES

Q11. During a traffic jam on a Highway, certain vehicles are given priority over other to give way. The possible types of vehicles on road and their associated priority are given below.

Vehicle Type	Priority
Emergency	10
Fire	9
School	8
President	7
Others	6

When a vehicle arrives, vehicle priority (an integer) is read and inserted into the traffic queue in the order of arrival. When the traffic is to be cleared, a vehicle with highest priority is allowed to move first and the vehicles following it in the queue move forward by one position. Write a program to simulate the above situation using suitable data structure. (4)

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

#define Max 50
```

```
enum code1{ hvy=1,prv=2,pub=2,govt,pol,mla,mp,pres,scl,fire,emr};

int q[10];
int r = -1;
int f=0;


// VEHICLES ENTER THE QUEUE IN THE ORDER IN WHICH THEY ENTER THE ROAD
void insert(int c)
{
  if(r==Max)
        printf("No space\n");
  else
        q[++r]=c;
}


//MOVE OUT THE VEHICLE WITH HIGHEST PRIORITY AND SHIFT THE VEHICLES BEHIND IT

int moved_out()
{
  int b=q[0],i,p=0;

  if(r<0)
    printf("No vehicle in the queue\n");
  else
  {
    for(i=0;i<=r;i++)
    {
      if(q[i]>b)
      {
```

```
            b=q[i];
           p=i;
            }
    }
    for(i=p;i<=r;i++)
            q[i]=q[i+1];
    r--;
    return b;
  }
}
int main()
{

   int c,code,d;
   char y;
   do
   {
      printf( " 1: Insert\t 2: Moved_out\t  3: CaptureTraffic \n");
      scanf("%d",&c);
      switch(c)
      {
            //READ THE VEHICLE CODE WHICH IS ALSO ITS PRIORITY
         case 1:printf("enter the vehical code\n");
              scanf("%d",&code);
              insert(code);
              break;
         case 2:d=moved_out();
               switch(d)
               {
               case hvy: printf("The vehicle moved is hvy\n");break;
               //case prv:
               case pub: printf("The vehicle moved is prv or pub\n"); break;
```

```
            case govt: printf("The vehicle moved is govt\n");   break;
            case pol: printf("The vehicle moved is pol\n");   break ;
            case mla: printf("The vehicle moved is mla\n");   break;
            case mp: printf("The vehicle moved is mp\n");    break;
            case pres: printf("The vehicle moved is preas\n");  break;
            case scl: printf("The vehicle moved is scl\n");break;
            case fire: printf("The vehicle moved is fire\n"); break;
            case emr: printf("The vehicle moved is emr\n");break;
            }
            break;
        case 3: CaptureTraffic();
            break;
    }
    printf(" Do u want to continue\n");
    scanf("%c",&y);

    scanf("%c",&y);
    }
    while(y=='Y'||y=='y');
    getch();
}
```

Q12. Consider a scenario of the billing counter of an XYZ company. There are two types of customers, i.e. premium(P) and loyal customers (L). As per the company policy, the premium customers are added to the front of the waiting line. If there are K premium customers at the front of the waiting line, all premium customers are moved to another line (Premium line). The loyal customers are moved to the front of the line as per their order. Due to the arrival and departure of customers, the waiting line will grow and shrink at any point of time. Always loyal customers are added to the end of the waiting line (original line). The customers leave the system as soon as they get the bill. Implement the scenario using an efficient data structure. (4)

Ex: Premium customer P Loyal customer L

K=3

After adding 2 Premium customers and 4 Loyal customers.

P→P→L→L→L→L Original line

The waiting line is shown below after three successive deletions

L→L→L

Original line

ADD 3 PREMIUM CUSTOMERS

P→P→P—L→L→L

After the insertion

Original line : P→P→P

Premium line : L→L→L

Structure declaration 0.5M
memory allocation 0.5M

```
q=(struct cust*)malloc(sizeof(struct cust));
```

INSERT FUNCTION

Inserting to original queue -1M

Inserting to Premium queue-1M

```
void ins_line()
{
    struct cust *temp=(struct cust*)malloc(sizeof(struct cust));
printf("enter Customer Type  1 for premium and 0 for loyal customers\n");
scanf("%d",&temp->cust_type);
if(temp->cust_type==1) c++;
temp->next=NULL;
if(c==3){
while(c>0){
        t=(struct cust*) malloc(sizeof(struct cust));
r=front;
t->cust_type=r->cust_type;
 t->next=NULL;
    if(front1==NULL)
```

```
        {front1=t;rear1=t;
        x=front;
        front=front->next;free(x);
        }
else
{
    rear1->next=t;
    rear1=t;
    x=front;
     front=front->next;
     free(x);
}
c--;}
return;
}
if(front==NULL)
{front=temp;
rear=temp;}
else if(temp->cust_type==0)
{
    rear->next=temp;
    rear=temp;
}
else if(temp->cust_type==1)
{           temp->next=front;
    front=temp;
}}
```

DELETE FUNCTION -1 M

```
void del()
{
struct cust *temp;
if(front==NULL&&front1==NULL)
```

```
{
    printf("Line is empty\n");
    return;
}
if(front1==NULL&&front!=NULL)
{

temp=front;
printf("the element to be deleted ---%c",temp->cust_type);
front=front->next;
if(front->cust_type==1)
    c--;
free(temp);

}
else
{
temp=front1;
printf("the element to be deleted ---%c",temp->cust_type);
front1=front1->next;
free(temp);
}
}
```

Q13. i) Write pseudo code to implement a queue data structure using an array , It should include the following operations

a) enqueue(item) b) dequeue() c) isEmpty() d) size()

ii)Write pseudo code for segregating prime numbers nodes in a Linked List using Dequeue

Original Linked List:

2 -> 3 -> 4 -> 5 -> 6 -> 7 -> None

Segregated Linked List (Prime numbers first):

7 -> 5 -> 3 -> 2 -> 4 -> 6 -> None

. (3)

enqueue(item) – 0.5M

dequeue() – 0.5M

isEmpty() – 0.5M

size() – 0.5M

prime number segregation- 1M

Basic Operations

```
struct Queue {
   int items[MAX_SIZE];
   int front;
   int rear;
   int size;
};
bool isEmpty(struct Queue *queue) {
   return queue->size == 0;
}
void enqueue(struct Queue *queue, int item) {
   if (queue->size == MAX_SIZE) {
      printf("Queue is full. Cannot enqueue.\n");
      return;
   }

   queue->rear = (queue->rear + 1) % MAX_SIZE;
   queue->items[queue->rear] = item;
   queue->size++;
```

```
}
int dequeue(struct Queue *queue) {
   if (isEmpty(queue)) {
      printf("Queue is empty. Cannot dequeue.\n");
      return -1; // Return a sentinel value to indicate an error
   }

   int item = queue->items[queue->front];
   queue->front = (queue->front + 1) % MAX_SIZE;
   queue->size--;
   return item;
}
int size(struct Queue *queue) {
   return queue->size;
}
```

Insertion

Initial Array

12, 45, 7, 63, 28, 90, 56

Step 1: To insert 99 at position 3, we need to shift the elements at and after position 3 one position to the right to make space for the new element.

12, 45, _, 7, 63, 28, 90, 56

Step 2: Now that we have created space at position 3, we can insert the element 99 into that position.

12, 45, 99, 7, 63, 28, 90, 56

Explanation: After the insertion, the element 99 is placed at position 3, and all the other elements to the right are shifted one position to the right. The array indices are adjusted accordingly.

Pseudo Code:

```
procedure insertElement(array, element, position)
   // Check if the position is valid
   if position < 0 or position > length(array) then
      return "Invalid position"
```

```
    // Make space for the new element
    for i from length(array) downto position + 1
        array[i] = array[i - 1]

    // Insert the new element at the specified position
    array[position] = element
end procedure
```

Q14. Write an algorithm for converting from prefix to postfix expression. Convert the following expression into postfix expression using stack (A+B)*C-D/E.. (3)

Algorithm: Prefix to Postfix Conversion

1. Initialize an empty stack for operators.
2. Initialize an empty string for the postfix expression.
3. Reverse the given prefix expression to process it from right to left.
4. Scan the reversed prefix expression character by character (from right to left).

   If the character is an operand, append it to the postfix expression.

   If the character is an operator (+, -, *, /)

   - Pop operators from the stack and append them to the postfix expression while the stack is not empty, and the operator at the top of the stack has higher or equal precedence compared to the current operator.
   - Push the current operator onto the stack.

   - If the character is an opening parenthesis '(', push it onto the stack.
   - If the character is a closing parenthesis ')':
   - Pop operators from the stack and append them to the postfix expression until an opening parenthesis '(' is encountered. Pop and discard the '('.
5. After scanning the entire expression, pop any remaining operators from the stack and append them to the postfix expression.
6. Reverse the postfix expression to get the final result.
7. The postfix expression is the desired output.

Conversion

Step-by-Step Conversion:

Initialize an empty stack and an empty postfix expression string.

Start scanning the infix expression from left to right:

(A + B) * C - D / E

Character by character, apply the rules of the algorithm:

(A + B) * C - D / E

(A + B) is an enclosed expression, so it's processed within parentheses:

AB+

* C - D / E

* has higher precedence than +, so it's pushed onto the stack:

AB+*

C - D / E

- has lower precedence than *, so * is popped from the stack and appended to the postfix expression, then - is pushed:

AB+*

C - D / E

C is an operand, so it's appended to the postfix expression:

AB+*C

- D / E

- is still in the stack, and it has higher precedence than D, so it's pushed:

AB+*C-

D / E

D is an operand, so it's appended to the postfix expression:

AB+*C-D

/ E

/ is pushed since it has higher precedence than -:

AB+*C-D/

E

E is an operand, so it's appended to the postfix expression:

AB+*C-D/E

After scanning the entire expression, pop any remaining operators from the stack and append them to the postfix expression:

Pop - and / from the stack and append them:

AB+*C-DE/

The postfix expression is now AB+*C-DE/

Q15. Write a program to sort a linked list using selection sort in ascending order. While creating the list add in the front only. Data that a linked list contains is a string.

Sample Input / Output

```
1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the node
zee
1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the node
bbb
1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the node
cat
1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the node
abc
1. Create  2. Sort   3. Diplay 4. Exit
3
abc
cat
bbb
zee
```

```
1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the
node
mmm
1. Create  2. Sort   3. Diplay 4. Exit
3
mmm
abc
cat
bbb
zee
1. Create  2. Sort   3. Diplay 4. Exit
2
1. Create  2. Sort   3. Diplay 4. Exit
3
abc
bbb
cat
mmm
zee
1. Create  2. Sort   3. Diplay 4. Exit
```

. (3)

```
#include <stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct list1 * ptr;
struct list1{
   char * str;
   ptr link;
};
typedef struct list1 listt;
```

```
void selectionsort(ptr *p)
{
   ptr small,temp,tri,trj,prev;
   int pos,i,j;
   for(tri=*p;tri!=NULL;tri=tri->link)
   {
      small=tri;
      //pos=0;

      for(trj=tri->link;trj!=NULL;trj=trj->link)
      {
         //prev=trj;
         if(strcmp(small->str,trj->str)>0)
         {
            small=trj;

         }
      }
        char temp[30];
        strcpy(temp,small->str);
        strcpy(small->str,tri->str);
        strcpy(tri->str,temp);
   }
   printf("inside sort\n");
}
void insertfront(ptr *p,char *s)
{

   ptr newnode=(ptr)malloc(1*sizeof(listt));
   newnode->str=malloc(30*sizeof(char));
   strcpy(newnode->str,s);
   printf("inside if %s ",newnode->str);
   if(*p==NULL)
   {
      printf("inside if if");
      newnode->link=NULL;
      *p=newnode;
   }
   else
   {

      newnode->link=*p;
      *p=newnode;
   }
   printf("hello");
}
void display(ptr p)
{
   ptr temp;
   while(p)
```

```
   {
      printf("%s\n",p->str);
      p=p->link;
   }

}

int main()
{
   printf("Hello World");
   ptr lst=NULL;
   int ch,count=0;;
   char *s=malloc(30*sizeof(char));
   do
   {
      printf("1. Create  2. Sort   3. Diplay 4. Exit\n");
      scanf("%d",&ch);
      switch(ch)
      {
         case 1: printf("enter the string to inserted to the node\n");
               scanf("%s",s);
               printf("%s ",s);
               insertfront(&lst,s);
               count++;
               printf(" hi %d",count);
               break;
         case 2: selectionsort(&lst);
               break;

         case 3 :display(lst);
            break;

      }
      printf("%c");
   }while(ch!=4);
   return 0;
}
Hello World1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the node
zee
zee inside if zee inside if ifhello hi 1�1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the node
bbb
bbb inside if bbb hello hi 2�1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the node
cat
cat inside if cat hello hi 3�1. Create  2. Sort   3. Diplay 4. Exit
1
```

```
enter the string to inserted to the node
abc
abc inside if abc hello hi 4�1. Create  2. Sort   3. Diplay 4. Exit
3
abc
cat
bbb
zee
1. Create  2. Sort   3. Diplay 4. Exit
1
enter the string to inserted to the node
mmm
mmm inside if mmm hello hi 5�1. Create  2. Sort   3. Diplay 4. Exit
3
mmm
abc
cat
bbb
zee
1. Create  2. Sort   3. Diplay 4. Exit
2
inside sort
1. Create  2. Sort   3. Diplay 4. Exit
3
abc
bbb
cat
mmm
zee
1. Create  2. Sort   3. Diplay 4. Exit
```

Scheme of evaluation

1. Creation of the list 1
2. Selection sort 1m
3. display and main 1m

Q16. Stacks can be implemented using linked lists with the help of dynamic memory allocation functions. Develop push() and pop() functions to implement a stack using linked list. The typical function calls would be push(&top, item) and item = pop (&top);. Assume the following initial declarations.

```
typedef struct node *Nodeptr;
struct node{
   int data;
   Nodeptr next;
};
int main()
{     Nodeptr top = NULL;   //stack is empty
```

```
    ...
}
```
. (3)

```
void push(Nodeptr *top, int item)
{
  /* add an element to the top of the stack */
  Nodeptr temp = (Nodeptr) malloc (sizeof (struct node));
  if (IS_FULL(temp))
  {
     printf(" The memory is full\n");
     exit(1);
     }
     temp->data= item;
     temp->next = *top;
     *top= temp;
}

int pop(Nodeptr *top)
{
  /* delete an element from the stack */
   Nodeptr temp = *top;
   int item;
   if (*top==NULL))
  {
     printf("The stack is empty\n");
     exit(1);
   }
    item = temp->data;
   *top = temp->link;
    free(temp);
    return item;
}
```

Q17. Illustrate the use of linked list to represent polynomials using structure. Specify the following C functions

i. Create N polynomials and take input for each of them.
ii. Identify the degree of polynomial and display that within the function. Polynomial is obtained as an argument. The highest power in a polynomial equation is considered as the degree of the polynomial. (3)

Answer:

Marks distribution

Structure + functions – 1M
i. N polynomials – pointer to N structures – dynamic memory allocation – 1M
ii. Degree calculation – 1M

Define a structure to represent individual terms of the polynomial, where each term has two components: the coefficient and the exponent. Then, you can use a linked list to connect these terms to form the polynomial.

```
struct Term {
    int coefficient;
    int exponent;
    struct Term* next;
};
```

struct Term - represent each term in the polynomial.
Each term has a coefficient and an exponent, as well as a pointer to the next term in the linked list.

```
// Function to create a new term
struct Term* createTerm(int coeff, int exp) {
    struct Term* newTerm = (struct Term*)malloc(sizeof(struct Term));
    if (newTerm == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }
    newTerm->coefficient = coeff;
    newTerm->exponent = exp;
    newTerm->next = NULL;
    return newTerm;
}

// Function to add a term to the polynomial (linked list)
void addTerm(struct Polynomial* poly, int coeff, int exp) {
    struct Term* newTerm = createTerm(coeff, exp);
    if (poly->head == NULL) {
        poly->head = newTerm;
    } else {
        struct Term* current = poly->head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = newTerm;
    }
}

// Function to display the polynomial
void displayPolynomial(struct Polynomial* poly) {
    if (poly->head == NULL) {
        printf("Polynomial is empty.\n");
        return;
    }
```

```
    struct Term* current = poly->head;
    while (current != NULL) {
        printf("%dx^%d", current->coefficient, current->exponent);
        if (current->next != NULL) {
            printf(" + ");
        }
        current = current->next;
    }
    printf("\n");
}
```

i. define a separate linked list for each polynomial.

```
// Define a structure for a polynomial
struct Polynomial {
    struct Term* head;
};
```

In main -

```
printf("Enter the number of polynomials (N): ");
    scanf("%d", &N);

    struct Polynomial* polynomials = (struct Polynomial*)malloc(N * sizeof(struct
Polynomial));

    for (int i = 0; i < N; i++) {
        polynomials[i].head = NULL;
                int numTerms;
printf("Enter the number of terms for Polynomial %d: ", i + 1);
        scanf("%d", &numTerms);
        for (int j = 0; j < numTerms; j++) {
            int coeff, exp;
            printf("Enter coefficient and exponent for term %d of Polynomial %d: ", j + 1, i + 1);
            scanf("%d%d", &coeff, &exp);
            addTerm(&polynomials[i], coeff, exp);
        }
    }

    // Displaying all the polynomials
    for (int i = 0; i < N; i++) {
        printf("Polynomial %d: ", i + 1);
        displayPolynomial(&polynomials[i]);
            }
```

1. Define a struct Polynomial to represent each polynomial.
2. Each polynomial has a head pointer, which points to the first term in its linked list of terms.
3. First input the number of polynomials (N) and then create an array of struct Polynomial to store these N polynomials.

4. For each polynomial, input the number of terms and then input the coefficient and exponent for each term, using the addTerm function to add them to the linked list of terms for that polynomial.
5. After inputting all polynomials, display each of them using the displayPolynomial function.

ii.

```
// Function to calculate and display the degree of the polynomial
void calculateAndDisplayDegree(struct Polynomial* poly) {
    if (poly->head == NULL) {
        printf("Polynomial is empty.\n");
        return;
            }
int degree = 0;
struct Term* current = poly->head;
    while (current != NULL) {
        if (current->exponent > degree) {
            degree = current->exponent;
        }
        current = current->next;
    }

    printf("Degree of the polynomial is: %d\n", degree);
            }
```

In main –

```
printf("Enter the number of polynomials (N): ");
    scanf("%d", &N);

    struct Polynomial* polynomials = (struct Polynomial*)malloc(N * sizeof(struct
Polynomial));

    for (int i = 0; i < N; i++) {
        polynomials[i].head = NULL;
        int numTerms;
                printf("Enter the number of terms for Polynomial %d: ", i + 1);
scanf("%d", &numTerms);
        for (int j = 0; j < numTerms; j++) {
            int coeff, exp;
            printf("Enter coefficient and exponent for term %d of Polynomial %d: ", j + 1, i + 1);
            scanf("%d%d", &coeff, &exp);
            addTerm(&polynomials[i], coeff, exp);
        }
        // Calculate and display the degree of the polynomial
        calculateAndDisplayDegree(&polynomials[i]);
    }
```

Q18.

Given a structure BOOK as follows:

typedef struct {char* title;char *author_name}BOOK;

Demonstrate the following using code snippets

i. Store N BOOKS using concept of pointers and dynamic memory allocation utilizing the above given structure BOOK
ii. Function to check if any two BOOKS are having same title and author. Pass entire structure as an argument. Display the result – Books are same or not – within the function. (2)

Answer:

i. Dynamic memory allocation + input for N books 1M
ii. Code snippet checking books to be same 1M

Define the BOOK structure as given.
Input the number of books (N) from the user and then create an array of pointers to BOOK structures (BOOK** books) to store N books.
Inside a loop, allocate memory for each book and its members (title and author name) dynamically.
Use malloc to allocate memory for the strings and copy the input values into them using strcpy.

```
// Define the BOOK structure
typedef struct {
    char* title;
    char* author_name;
} BOOK;

int main() {
    int N; // Number of books
    printf("Enter the number of books (N): ");
    scanf("%d", &N);

    // Create an array of pointers to BOOK structures
    BOOK** books = (BOOK**)malloc(N * sizeof(BOOK*));

    // Input details for each book
    for (int i = 0; i < N; i++) {
        books[i] = (BOOK*)malloc(sizeof(BOOK));

// Input title
        char title[100];
        printf("Enter the title of Book %d: ", i + 1);
        scanf(" %[^\n]", title);
        books[i]->title = (char*)malloc(strlen(title) + 1);
        strcpy(books[i]->title, title);

// Input author name
        char author[100];
        printf("Enter the author name of Book %d: ", i + 1);
```

```
        scanf(" %[^\n]", author);
        books[i]->author_name = (char*)malloc(strlen(author) + 1);
        strcpy(books[i]->author_name, author);
    }
```

ii.

Define a areBooksSame function that takes the array of BOOK structures and the number of books (N) as arguments.
This function checks if any two books have the same title and author by comparing each book with every other book in a nested loop.
Inside the main function, call the areBooksSame function and display the result.

```
// Function to check if any two books have the same title and author
int areBooksSame(BOOK** books, int N) {
    for (int i = 0; i < N - 1; i++) {
        for (int j = i + 1; j < N; j++) {
            if (strcmp(books[i]->title, books[j]->title) == 0 &&
                strcmp(books[i]->author_name, books[j]->author_name) == 0) {
                return 1; // Books are the same
            }
        }
    }
    return 0; // No two books are the same
}
```

In main -

```
// Check if any two books are the same
    int result = areBooksSame(books, N);

    // Display the result
    if (result) {
        printf("At least two books are the same (with the same title and author).\n");
    } else {
        printf("No two books are the same (with the same title and author).\n");
    }
```